Development, Evaluation and Reflection on BFRdist

DATA501 Package Development Report

Daniel Luo and Jennifer Hanna | Package snapshot: BFRdist v1.0.1

Executive summary

This report reflects on the development of BFRdist, an R package built to make the BFR probability distribution usable primarily in applied analysis of Generalized Radar Vegetation Index (GRVI) measurements. The project consists of the standard distribution-function interface, a compiled C++ initial estimator, numerical fitting, descriptive statistics, S3 methods, a diagnostic bootstrap plot, documentation, a vignette, and a unit testing suite leveraging *testthat*.

Overall, the quality and production process of the BFRdist library was quite strong, having successfully captured the statistical applications of the BFR distribution. The test suite is quite robust, and the vignettes are well documented. There are example applications using both synthetically generated data, as well as a test sample of actual agricultural data from which the actual BFR distribution was designed for (Bhattacharya, 2026).

However, there are still points of improvement that could have been made during the production process, because of external factors such as time constraints or early misinterpretations of the package's theoretical functionality. Mapping out a coherent strategy and relying on pre-planned steps rather than developing in a reactionary manner is something I would try to do if I was to start from scratch.

1. Project context and development approach

The following describes in detail how the different components of the project were developed, in chronological order. The work was about evenly distributed between me and Jennifer. Below are the data flow and sequence diagrams of the package:

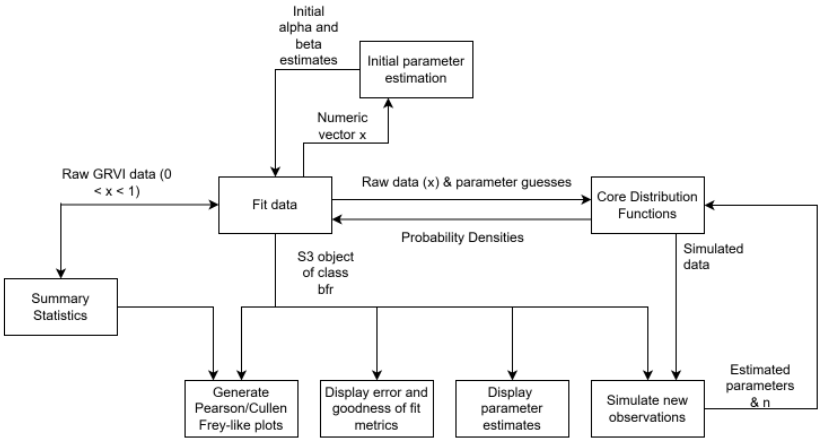


Figure 1: Data flow diagram illustrating the parameter estimation pipeline.

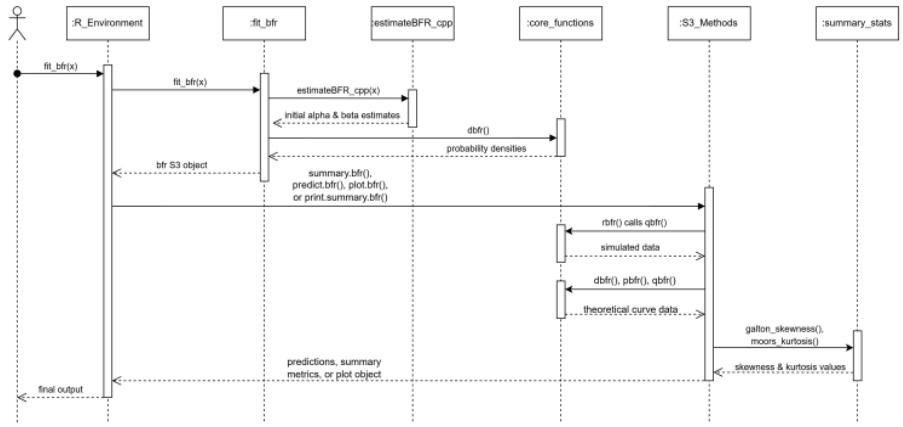


Figure 2: Sequence diagram of the s3 object generation and method dispatch processes.

1.0 Development etiquette

Using my software engineering experience, I made sure that all the following steps were encapsulated inside feature branches to allow parallel development without issue. This allowed for smooth peer reviewed merge requests into the main branch, rather than making direct commits to the main branch. A clear commit history allows for easy backtracking and debugging if required, and this was made easier by retaining the feature branches instead of deleting them as well as squashing commit messages during merge requests. Clear commit messages that describe the exact issue solved in the commit were implemented, as well as making sure that only targeted changes are committed each time. Tags roughly follow semantic versioning naming conventions.

1.1 Establishing the distribution layer

The first foundational task was translating the mathematical definition into the standard R dpqr setup: *dbfr*() for density, *pbfr*() for cumulative probability, *qbfr*() for quantiles, and *rbfr*() for random generation. The quantile function also supports inverse-transform random generation, allowing simulation without requiring a separate sampling algorithm.

The implementation includes checks for positive shape parameters and values strictly inside (0, 1). These checks make the distribution domain explicit and prevent invalid inputs from silently producing misleading numeric results. The tests then reinforce the intended contract by checking boundary values, missing and infinite values, and invalid shape parameters.

Deriving the implementation of these mathematical functions in code form was mostly delegated to Jennifer, as to better leverage her mathematical background compared to mine, while I implemented the quantile estimator.

1.2 Adding parameter estimation

The package extends the distribution functions with *estimateBFR*(), *fit_bfr*(), and two estimation approaches: maximum likelihood estimation (MLE) and the method of moments (MME). The R wrapper validates input before calling a compiled C++ routine that provides quartile-based initial estimates. Those estimates are then used as starting values for *stats::optim*() with bounded optimisation. This setup separates a fast starting-value calculation from the more general numerical fitting stage.

The C++ components were implemented by me as part of the backend development process. Pros of implementing C++ here is that we get good initial values that can help an optimiser converge more consistently, and a compiled implementation can be useful for computationally intensive calculations. However, using C++ increases build and debugging complexity by a small amount. This in my opinion is a beneficial enough trade for the purpose of the package.

1.3 Turning estimates into usable model objects

fit_bfr() returns an object with class bfr, containing parameter estimates, the method used, the original data, and the optimiser result. The package supplies S3 methods for printing, summarising, predicting, and plotting these objects. This makes the output more useful than a bare numeric vector: users can inspect fit statistics, generate new observations, and produce a diagnostic display through familiar R generic functions.

1.4 Utilising model objects as part of visualisation

A key part of BFRdist's capabilities involves using Galton skewness and Moors kurtosis, which are based on quantiles rather than central moments. This in turn supports the SK_G × KU_M diagnostic plot. The plot uses parametric bootstrap replicates and confidence ellipses to compare the observed statistics with those expected under the fitted model.

This portion of the package was primarily implemented by me. The SK_G x KU_M diagnostic plot was designed to mimic/resemble the example presented in the original journal where the BFR Distribution was originally presented (Frery, 2026). This is to reflect the data context for which the BFR distribution was designed for.

2. Testing and validation

The testthat suite is organised into named test_that() blocks, uses fixed seeds for reproducibility, and checks both ordinary usage and deliberately invalid inputs. The tests document intended behaviour and provide a safety net of a baseline when implementation details change.

2.1 A summary of the tests

- Domain and parameter validation: tests check that values at or outside the unit-interval boundaries are rejected, alongside NA/Inf inputs and non-positive shape parameters.
- Mathematical consistency: tests verify non-negative density values, CDF values within [0, 1], a strictly increasing CDF over a test grid. The tests also check that the round-trip relationship *pbfr*(*qbfr*(p)) ≈ p.
- Simulation and estimation: tests check random sample length and support, fit MLE and MME models, and verify that MLE estimates from a large synthetic sample are reasonably close to known generating parameters.
- S3 interface: tests check the bfr and summary.bfr classes, parameter names and types, prediction length and support, and expected console output from print methods.
- Error handling and edge cases: unsupported estimation methods, constant data, invalid prediction horizons, missing-value removal in robust statistics, and direct C++ wrapper validation all receive attention.

- Applied data: tests check that exact zero values in the GRVI data are rejected, and that cleaned Canola measurements can be fitted. This also tests the pre-existing example presented in the vignette about how to use the sample of real-life data.

3. Documentation, vignette and reproducibility

The BFRDist package uses roxygen2-style comments. This is a maintainable approach because documentation can live close to implementation and be regenerated. The DESCRIPTION file records imports, linking to Rcpp, suggested test and vignette dependencies, the vignette builder, and the R version requirement. NAMESPACE and Rcpp export files are also included.

The vignette, "Introduction to BFRdist", is a key part of the user experience. It introduces the mathematical formulation, shows how to install and load the package, demonstrates the core functions, generates synthetic observations with known parameters, and applies the package to the included GRVI data. It also shows fitting, summary, prediction, and diagnostic plotting. This end-to-end sequence is more useful than isolated examples because it models a realistic analysis workflow.

The vignette also explains a data constraint: exact zero and one values must be filtered before fitting because the model is defined on the open interval (0, 1). The tests reinforce this point by using a real-data sample of GRVI_DpRVI data.

In general, the vignette provides objective definitions to the scope of the package, the context of the package's usage, which in this case is primarily in the field of agriculture. It also highlights limitations of its usage both mathematically as elaborated above, as well as warning the user about long computation times when using the MLE functionality. An acknowledgement of the foundational paper as well as a brief list of pros and cons are found in the conclusion and acknowledgements section of the vignette.

The README and peer assessment plan offer installation and testing guidance. Release checks verify that package installation, tests, documentation generation, and vignette building all work together in a clean environment.

4. Peer Review and Release

Once the unit tests, vignettes and basic functionality of the package were established, the final phase of the development cycle began. We performed bidirectional peer reviews with other members of the class, which served as a way of quality assurance. This process allowed for the discovery of a few build bugs and allowed us to patch them before the final release, which was crucial in ensuring the quality of the finished product. This allowed for a very smooth final package release and adequate amount of time to reflect on the positives as well as the shortcomings of the whole process.

5. Conclusion and self-reflection:

While I have a strong background in software development, transitioning from working primarily in backend development of SAAS products to a very statistics-heavy assignment was quite a challenge. The final BFRDist product came out successful, and the development journey helped to highlight areas of further growth in my skillset.

One of the significant learning curves of the project was perfectly aligning the software functionality with the BFR mathematical definitions. While Jennifer handled most of the mathematical side to allow me to handle the development of S3 functionalities and the other more code-heavy sections, it meant that I could observe her development process and improve my understanding of the theoretical side of the project.

One area of theoretical development that I observed was the implementation of the estimators. Here, a decision was made to pick the better estimator for the task at hand, and that meant pivoting from trying to force an MME implementation to instead rely on Quantile-based estimators and Maximum Likelihood estimation. This was because the BFR Distribution does not possess a general closed form expression for its k-order moments across the parameter space (Bhattacharya, 2026).

On the other hand, working in C++ was a challenge despite my Object-Oriented Programming background, as I had never previously worked with C++ as a language. The development process required cross-referencing code from varying sources, ranging from public GitHub repositories to examples from CRAN. CRAN examples were the most helpful as Rcpp is a common method of integrating both C++ and R functionalities.

If I were to start again from scratch, I would try to create a more cohesive and solid plan to map out the development cycle effectively, without having to make reactionary decisions. A solid strategy for visualising the architectural planning of the project would be to create a mind map (Frery, 2020) rather than a simple text-based proposal. Visualisation helps to view a problem from the high level, and allow for low level tasks/issues to becomge predictable and manageable. A diagram of the mind map is featured below:

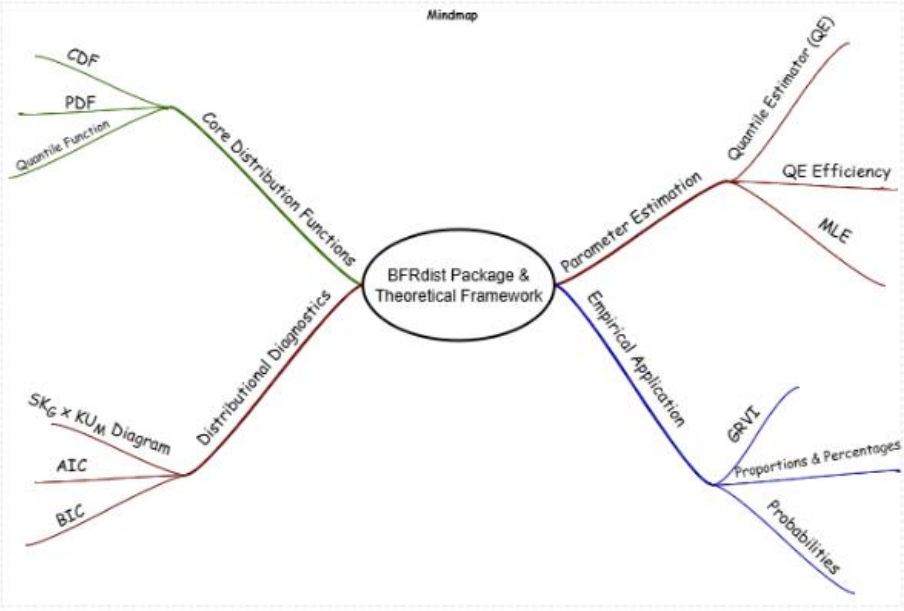


Figure 3: architectural mind map

In general, the BFRDist package was a success that achieves its goals. The final implementation of the code applies the theoretical basis behind the BFR distribution as defined in (Frery, 2026) quite well, an also handles the transition of raw bounded data into a sterile *fitBFR*() environment. A compiled C++ backend allowed for powerful implementation of the package's custom methods, such as print, summary, predict and plotting an SK_G x KU_M graph, alongside returning the BFR S3 object to the user. The uniqueness of this project is novel and quite different to what I have worked on thus far, being that it showcases a powerful meld between applied statistics and agricultural data analysis. The finished product is a highly functional statistical tool, and tested the application of my existing skills in a different context, while also improving the development of my statistical skillset.

6. References:

[1] Frery, A., & Bhattacharya, A. (2026). A Statistical Framework for Bounded Measurements with Applications to SAR Data. IEEE Transactions on Geoscience and Remote Sensing

[2] Bhattacharya, A., & Frery, A., & Rey, A. (2026) GRVI Data.

[3] Frery, A., C., Gomez, L., & Medeiros, A. C. (2020). A Badging System for Reproducibility and Replicability in Remote Sensing Research. IEEE Journal of Selected Topics on Applied Earth Observations and Remote Sensing, 13, 4988-4995. https://doi.org/10.1109/JSTARS.2020.3019418